

Implemented Code (C++): (Fractional Knapsack)

```
#include<bits/stdc++.h>
using namespace std;
using vec_1D=vector<double>;
using Pvec_1D=vector<pair<double,int>>;
#define newl cout<<endl

Pvec_1D Ratio;
double R[1000];
Pvec_1D Merge(Pvec_1D left,Pvec_1D right)
{
    int n=left.size()+right.size(),l=0,r=0;
    Pvec_1D ans(n);
    for(int i=0;i<n;i++)
    {
        if(l!=int(left.size())&&left[l].first<=right[r].first||r==int(right.size()))
        {
            ans[i].first=left[l].first;
            ans[i].second=left[l].second;
            l++;
        }
        else if(r!=int(right.size())&&right[r].first<=left[l].first||l==int(left.size()))
        {
            ans[i].first=right[r].first;
            ans[i].second=right[r].second;
            r++;
        }
    }
    return ans;
}

Pvec_1D Merge_Sort(int l, int h, Pvec_1D arr)
{
    if(l==h)
    {
        return {make_pair(arr[l].first,arr[l].second)};
    }
    int mid=(l+h)/2;
    Pvec_1D left=Merge_Sort(l,mid,arr);
    Pvec_1D right=Merge_Sort(mid+1,h,arr);
    return Merge(left,right);
}

double Frac_Knapsack(double capacity,vec_1D weight,vec_1D benefit)
{
    double total_benefit=0.0;
    for(auto i:Ratio)
```

```

{
    if (capacity >= weight[i.second])
    {
        total_benefit += benefit[i.second];
        capacity -= weight[i.second];
        R[i.second] = 1;
    }
    else if (capacity < weight[i.second])
    {
        total_benefit += (capacity / weight[i.second]) * benefit[i.second];
        R[i.second] = (capacity / weight[i.second]);
        capacity = 0;
        break;
    }
}
return total_benefit;
}
int main()
{
    ifstream ip;
    ip.open("knapsack.txt");
    int n;
    ip >> n;
    vec_1D weight(n), benefit(n);
    cout << "P/W: ";
    for (int i = 0; i < n; i++)
    {
        ip >> weight[i] >> benefit[i];
        double k = benefit[i] / weight[i];
        Ratio.push_back({-k, i});
        cout << k << " ";
    }
    Ratio = Merge_Sort(0, n - 1, Ratio);
    newl; newl;
    double capacity;
    ip >> capacity;
    double value = Frac_Knapsack(capacity, weight, benefit);
    cout << "Total Profit: " << value; newl; newl;
    cout << "Ratio: ";
    for (int i = 0; i < n; i++)
    {
        cout << R[i] << " ";
    }
    newl;
}

```

Output:

```
P/W: 1.5 1.25 1.6 1.33333 1.11111
Total Profit: 26.6667
Ratio: 1 1 1 1 0.666667
Press any key to continue . . .
```

Implemented Code (C++): (Job Sequence)

```
#include<bits/stdc++.h>
using namespace std;
using vec_1D=vector<double>;
using Pvec_1D=vector<pair<double,ll>>;
#define newl cout<<endl

Pvec_1D high;
vec_1D job(100);
vec_1D job_sec(100);
Pvec_1D Merge(Pvec_1D left,Pvec_1D right)
{
    int n=left.size()+right.size(),l=0,r=0;
    Pvec_1D ans(n);
    for(int i=0;i<n;i++)
    {
        if(l!=int(left.size())&&left[l].first<=right[r].first||r==int(right.size()))
        {
            ans[i].first=left[l].first;
            ans[i].second=left[l].second;
            l++;
        }
        else if(r!=int(right.size())&&right[r].first<=left[l].first||l==int(left.size()))
        {
            ans[i].first=right[r].first;
            ans[i].second=right[r].second;
            r++;
        }
    }
    return ans;
}

Pvec_1D Merge_Sort(int l, int h, Pvec_1D arr)
{

```

```

    if(l==h)
    {
        return {make_pair(arr[l].first,arr[l].second)};
    }
    int mid=(l+h)/2;
    Pvec_1D left=Merge_Sort(l,mid,arr);
    Pvec_1D right=Merge_Sort(mid+1,h,arr);
    return Merge(left,right);
}
double Frac_Knapsack(vec_1D profit,vector<int>deadline,int slot)
{
    double total_benefit=0.0;
    double time=0;
    for(auto i:high)
    {
        int temp=deadline[i.second];
        if (job_sec[temp]==0)
        {
            job[i.second]=1;
            job_sec[temp]=i.second+1;
            total_benefit+=profit[i.second];
            time++;
        }
        else
        {
            temp--;
            while(temp>0)
            {
                if(job_sec[temp]==0)
                {
                    job[i.second]=1;
                    job_sec[temp]=i.second+1;
                    total_benefit+=profit[i.second];
                    time++;
                    break;
                }
                temp--;
            }
        }
        if(time==slot)
        {
            break;
        }
    }
    return total_benefit;
}
int main()

```

```

{
    ifstream ip;
    ip.open("job.txt");
    int n;
    ip>>n;
    vec_1D profit(n);
    vector<int>deadline(n);
    cout<<"(J,D): ";
    int slot=-10;
    for(int i=0;i<n;i++)
    {
        ip>>profit[i]>>deadline[i];
        high.push_back({-profit[i],i});
        slot=max(slot,deadline[i]);
        cout<<"("<<profit[i]<<","<<deadline[i]<<") ";
    }
    high=Merge_Sort(0,n-1,high);
    newl;newl;
    double value=Frac_Knapsack(profit,deadline,slot);
    cout<<"Total Profit: "<<value;newl;newl;
    cout<<"Job: ";
    for(int i=0;i<n;i++)
    {
        cout<<job[i]<<" ";
    }
    newl;newl;
    cout<<"Job Sequence: ";
    for(int i=1;i<=slot;i++)
    {
        cout<<job_sec[i]<<" ";
    }
    newl;
}

```

Output:

```

(J,D): (100,2) (10,1) (15,2) (27,1)

Total Profit: 127

Job: 1 0 0 1

Job Sequence: 4 1

Press any key to continue . . . |

```